Business Management System
Software Design Description (SDD)

Section 2: Design Considerations

2. Design Considerations

2.1 Design Goals

The software design of the Business Management System (BMS) is governed by a set of foundational technical objectives. These design goals ensure that the resulting system meets production standards for enterprise software, maximizing lifecycle efficiency and minimizing long-term engineering overhead.

High Availability and Fault Tolerance

The system architecture must maintain an annual uptime profile of 99.95%, excluding scheduled maintenance windows. This goal requires the design to completely isolate system components so that the failure of an auxiliary module—such as reporting or digital receipt generation—does not impact core transactional capabilities like Point of Sale operations or database persistence. Fault tolerance is reinforced at the software level through structured exception filtering, asynchronous fallback queues, and multi-node service redundancy.

Transactional Integrity and Strict Data Consistency

Because the system orchestrates financial data, inventory valuation ledger records, and legal tax compliance entries, the architecture enforces absolute transactional atomicity, consistency, isolation, and durability (ACID). Every state adjustment across product metrics, inventory counts, or sales values must be evaluated in isolated, database-level transactions. The system prevents race conditions, write-skew anomalies, and phantom reads under concurrent multi-user environments by utilizing strict PostgreSQL isolation levels and row-level locking patterns.

Low-Latency Operational Performance

The Point of Sale user interface demands near-instantaneous query feedback to ensure high operational throughput at physical checkout points. The backend engineering target mandates that 95% of standard transactional operations (API read/write cycles) must execute with an end-to-end network round-trip time under 200 milliseconds. This performance threshold is achieved through database index optimization, aggressive Redis in-memory key-value caching, optimized raw database queries via the ORM layer, and Next.js server-side component streaming.

Seamless Horizontal and Vertical Scalability

The underlying computing topology must support seamless scalability to handle growing numbers of multi-tenant businesses, physical retail branches, and concurrent customer checkouts. The system avoids tightly coupled execution dependencies so that stateless application container layers can scale horizontally across distributed cloud environments via orchestrators. The persistence tier is designed for rapid vertical scale-ups and reads are offloaded using dedicated replication clusters.

Extensible Modular Customization

To ensure the system can accommodate future business needs without requiring complete core refactoring, the software design uses modular design patterns. Domain contexts—such as inventory, billing, or access control—are decoupled into isolated packages that interact through strict, contractually defined internal interfaces. This clear separation of concerns ensures that future feature expansions, such as integrating AI sales forecasting or automated accounting, can be added with minimal impact on existing code structures.

2.2 Assumptions

The architectural patterns and technical decisions specified throughout this design document depend on several contextual baseline assumptions. Should any of these assumptions be violated during implementation or production deployment, parts of the system design may require revisions.

Persistent Network Interconnectivity

It is assumed that target small and medium enterprises deploy this platform within environments supported by high-speed, stable broadband internet connectivity. The core Point of Sale and inventory tracking workflows are designed as cloud-native applications that require uninterrupted, low-latency access to the centralized backend NestJS application cluster and the PostgreSQL database engine.

Modern Browser Runtime Execution

The client application environment is assumed to run on modern, standard internet browsers that fully support the ECMAScript 6+ standard, hardware-accelerated CSS rendering, advanced DOM manipulation models, and secure cryptographic storage mechanisms (such as SessionStorage and local Cookie policies). Supported runtimes include current enterprise versions of Google Chrome, Mozilla Firefox, Microsoft Edge, and Apple Safari. Legacy web engines lacking modern asynchronous processing capabilities are excluded from target compatibility metrics.

Centralized Singly Rooted Database Persistence

The architecture assumes that all tenant data across all organizational profiles can be securely isolated and maintained within a single, highly optimized relational PostgreSQL database cluster using strict logical filtering (Tenant-ID separation columns). While the design accommodates independent read-replicas for analytics, it assumes that a single, unified database schema serves as the single source of truth for all transactional writes.

2.3 Constraints

The implementation of the Business Management System is bounded by concrete technical, operational, regulatory, and architectural constraints that limit design flexibility but guarantee system security and consistency.

Hardware and Memory Limits

The application servers are configured to execute inside isolated Linux Docker containers limited to standard budget cloud instance allocations. The backend runtime execution space is restricted to a maximum of 2 Gigabytes of RAM per container node, while the Redis cache instance operates under strict memory eviction rules (Least Recently Used policy) with an upper memory threshold of 1 Gigabyte. The database schema design must minimize row overhead and use optimized data types to avoid excessive memory consumption.

Relational Engine Version Bounds

The data persistence layer is strictly bound to PostgreSQL version 18.0 or higher. The design relies on specific modern relational engine capabilities, including advanced B-Tree index optimizations, jsonb structural manipulation primitives, and optimized multi-table common table expressions (CTEs). The architecture cannot be backward-migrated to legacy SQL environments or switched to non-relational document databases without invalidating the core transaction design.

Data Privacy and Security Regulations

The system must comply with data protection regulations regarding the processing, retention, and encryption of commercially sensitive merchant records, customer personal identifying information (PII), and financial history logs. These rules mandate strict constraints on the persistence layer: password hashes cannot be retrievable in plaintext, all session payloads must be signed using private asynchronous keys, and comprehensive database audit logs must capture every write modification.

2.4 Design Principles

To maintain high code quality and clear separation of concerns across both frontend and backend modules, the system architecture adheres to several industry-standard design principles.

SOLID Object-Oriented Design Principles

All classes, modules, and providers within the NestJS and Next.js applications follow the foundational SOLID principles:

Single Responsibility Principle (SRP): Each class or service is responsible for exactly one business function (e.g., the ReceiptGenerationService handles formatting and streaming data, but does not process payments).

Open/Closed Principle (OCP): Application modules are open for extension but closed for modification, utilizing abstract interfaces and dependency injection.

Liskov Substitution Principle (LSP): Subtypes can substitute for their base types without altering system correctness.

Interface Segregation Principle (ISP): Clients are not forced to depend on extensive interfaces they do not use; domain boundaries use concise, specialized contracts.

Dependency Inversion Principle (DIP): High-level business logic modules do not import low-level data access layers directly; both depend on abstract contracts, implemented via NestJS's dependency injection engine.

Don't Repeat Yourself (DRY)

The codebase avoids duplicate code structures by encapsulating cross-cutting workflows into reusable utility layers, custom middleware, and shared hooks. Common procedures, such as calculating product taxes, verifying permission grants, or formatting currency strings, are declared once within shared libraries and referenced uniformly across all domain modules.

Separation of Concerns (SoC)

The application architecture divides responsibilities across distinct operational layers:

[Presentation Layer: Next.js + Tailwind CSS]

■

▼ (REST API / JWT Auth)

[Controller/Routing Layer: Request Validation]

■

▼

[Business Logic Layer: NestJS Services & Domain Rules]

■

▼

[Persistence Layer: Prisma ORM + PostgreSQL 18]

This strict layering prevents database entities from leaking into presentation views and ensures that user interface state alterations cannot bypass backend business validation rules.

2.5 Architectural Patterns

The complete system is built around standard, enterprise-proven architectural patterns that balance performance with modular maintainability.

Monolithic Modular Backend Architecture

The backend uses a modular monolithic architecture. Unlike decoupled microservices, which add significant network overhead and deployment complexity, all domains are compiled into a single executable runtime unit. However, logical modularity is strictly enforced: domains like UserModule, InventoryModule, and SalesModule maintain internal encapsulation, communicating exclusively via injected services. This design provides a clean migration path to an independent microservices architecture if transaction volumes eventually require physical service separation.

Model-View-Controller (MVC) and Layered Domain Architecture

The presentation and interaction models rely on a modernized variant of the layered MVC pattern. The Next.js frontend acts as an active presentation view that binds data fields to structural form models. On the server side, NestJS Controllers intercept incoming network payloads, delegate business operations to Domain Services, and interact with the database using Prisma ORM Data Models.

Repository and Data Mapper Patterns

To decouple business logic from raw SQL query execution, the database tier uses data mapper abstractions managed by Prisma ORM. Database schemas are compiled into strongly typed application objects. Complex querying operations that require advanced performance optimization are isolated within specialized repository classes, shielding the service layer from schema modification risks.

2.6 Technology Selection Rationale

Each component within the chosen technology stack was selected to satisfy specific operational requirements, development velocities, and performance targets.

■■
■■■■■■

■ BMS ADVANCED TECHNOLOGY STACK ■

■■
■■■■■■

■ FRONTEND TIER ■ BACKEND TIER ■

■ • Next.js (App Router) ■ • NestJS Framework ■

■ • TypeScript ■ • Prisma ORM ■

■ • Tailwind CSS + Shadcn UI ■ • TypeScript Runtime ■

■■
■■■■■■

■ PERSISTENCE, CACHING & INFRASTRUCTURE TIER ■

■ • PostgreSQL 18 (ACID Core) ■ • Redis (Session & Cache) ■

■ • Nginx (Reverse Proxy) ■ • Docker & Docker Compose ■

■■
■■■■■■

Frontend Framework: Next.js, Tailwind CSS, and Shadcn UI

Next.js provides an optimal frontend architecture by merging server-side rendering performance with reactive client-side interfaces. Server Components fetch initial layouts directly within the cloud infrastructure, significantly speeding up First Contentful Paint metrics. Tailwind CSS ensures low asset sizes by generating atomic utility styles, avoiding the bloat of traditional UI frameworks. Shadcn UI provides unstyled, accessible primitives that give engineers complete control over DOM behavior, which is essential for building a fast, accessible Point of Sale interface.

Backend Engine: NestJS and TypeScript

NestJS provides a highly structured engineering architecture for Node.js environments. By enforcing typescript development out of the box, it ensures complete type safety from backend database access points all the way to API contracts. NestJS's powerful modular dependency injection system simplifies testing and makes it easy to replace mock implementations with real services. This allows teams to build highly maintainable codebases that can scale alongside growing business logic complexity.

Persistence Engine: PostgreSQL 18 and Prisma ORM

PostgreSQL 18 offers advanced relational capabilities, reliable ACID compliance, and excellent connection handling under heavy concurrent write loads. Prisma ORM translates these capabilities into the application space by auto-generating typescript interfaces directly from the database schema definition. This setup catches structural type mismatches during compilation rather than at runtime, while still allowing developers to write optimized, high-performance raw SQL queries for complex analytical reporting when needed.

Performance Caching & Infrastructure: Redis, Nginx, and Docker

Redis acts as an in-memory database that handles high-throughput session cache states, short-lived API responses, and rate-limiting counters, offloading repetitive query stress from the relational database. Nginx sits at the edge of the architecture, serving as a high-performance reverse proxy that manages TLS/SSL termination, applies security headers, and decompresses static client assets. Finally, Docker and Docker Compose wrap every service into consistent, isolated software containers, ensuring predictable behavior and identical runtime environments from local development all the way to production clusters.